

step02_02_ETL_cleaning

October 6, 2026

```
[1]: import duckdb
import json
import pandas as pd
from pathlib import Path

pd.set_option("display.width", 220)
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_rows", 120)

BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data_sets"
RESULT_DIR = BASE_DIR / "results"
RESULT_DIR.mkdir(exist_ok=True)
TMP_DIR = RESULT_DIR / "duckdb_tmp"
TMP_DIR.mkdir(exist_ok=True)

con = duckdb.connect(str(RESULT_DIR / "step01_03_5.duckdb"))
con.execute("SET memory_limit='6GB'")          # lower this if your laptop
↳ has < 8 GB RAM
con.execute(f"SET temp_directory='{TMP_DIR.as_posix()}'" )
con.execute("SET preserve_insertion_order=false")

FINDINGS = {}          # everything that matters is collected here -> saved as
↳ JSON at the end

def q(sql):
    return con.execute(sql).df()

def save(df, name):
    df.to_csv(RESULT_DIR / name, index=False)
    print(f"    -> saved results/{name}")

[2]: # ## 0. Load tables into DuckDB (delay is loaded once and cached in the .duckdb
↳ file)
```

```

def load(name, fname, types=None):
    path = (DATA_DIR / fname).as_posix()
    t = f", types={types}" if types else ""
    con.execute(
        f"CREATE OR REPLACE TABLE {name} AS "
        f"SELECT * FROM read_csv('{path}', header=true{t})"
    )

load("train_details", "train_details.csv", {"train_no": "BIGINT"})
load("station_full_names", "station_full_names.csv")
load("schedule", "combined_schedule.csv", {"train_no": "BIGINT",
↪ "station_no": "INTEGER"})

already = con.execute(
    "SELECT count(*) FROM information_schema.tables WHERE table_name='delay'"
).fetchone()[0]
if not already:
    print("Loading combined_delay.csv (one-time, a few minutes)...")
    load("delay", "combined_delay.csv",
        {"date": "DATE", "station_no": "INTEGER", "delay": "DOUBLE",
↪ "train_no": "BIGINT"})

for t in ["train_details", "station_full_names", "schedule", "delay"]:
    print(t, con.execute(f"SELECT count(*) FROM {t}").fetchone()[0])

# Helper tables used repeatedly
con.execute("CREATE OR REPLACE TABLE sched_stations AS SELECT DISTINCT
↪ station_name FROM schedule")
con.execute("CREATE OR REPLACE TABLE delay_stations AS SELECT DISTINCT
↪ station_name FROM delay")

```

```

train_details 8992
station_full_names 8963
schedule 172112
delay 38428703

```

[2]: <_duckdb.DuckDBPyConnection at 0x2f082fe22f0>

```

[3]: print("=== STEP 1: CANCELLATION DROP RULE ===")

# 1. Calculate journey statistics to find phantom trains
con.execute("""
CREATE OR REPLACE TEMP TABLE JourneyStats AS
SELECT
    date,
    train_no,

```

```

        COUNT(delay) AS valid_delays,
        COUNT(*) AS total_route_stops
FROM delay
GROUP BY date, train_no
""")

# 2. Print a sample of the completely missing journeys being dropped
dropped_sample = q("""
    SELECT date, train_no, total_route_stops, valid_delays
    FROM JourneyStats
    WHERE valid_delays = 0
    LIMIT 5
""")
print("\nSample of Dropped Cancelled Journeys (0 valid delays logged):")
print(dropped_sample.to_string(index=False))

# 3. Execute the Drop Rule to create the working table
con.execute("""
CREATE OR REPLACE TABLE delay_step1 AS
SELECT d.*
FROM delay d
JOIN JourneyStats j ON d.date = j.date AND d.train_no = j.train_no
WHERE j.valid_delays > 0
""")

# 4. Extract and print Step 1 Metrics
metrics_1 = q("""
    SELECT
        (SELECT COUNT(*) FROM delay) AS original_rows,
        (SELECT COUNT(*) FROM delay_step1) AS surviving_rows,
        (SELECT COUNT(*) FROM delay WHERE delay IS NULL) AS original_nulls,
        (SELECT COUNT(*) FROM delay_step1 WHERE delay IS NULL) AS_
    remaining_nulls
""")

orig_rows = int(metrics_1['original_rows'][0])
surviving_rows = int(metrics_1['surviving_rows'][0])
orig_nulls = int(metrics_1['original_nulls'][0])
rem_nulls = int(metrics_1['remaining_nulls'][0])

print("\n--- STEP 1 METRICS ---")
print(f"Original Records: {orig_rows:,}")
print(f"Records Dropped (Cancelled Trains): {orig_rows - surviving_rows:,}")
print(f"Null Delays Eliminated via Drop: {orig_nulls - rem_nulls:,}")
print(f"Null Delays Remaining to Impute: {rem_nulls:,}")

```

=== STEP 1: CANCELLATION DROP RULE ===

Sample of Dropped Cancelled Journeys (0 valid delays logged):

date	train_no	total_route_stops	valid_delays
2026-02-01	54015	7	0
2025-02-10	53214	31	0
2025-02-22	53214	31	0
2025-02-19	18417	37	0
2025-02-12	55031	25	0

```
FloatProgress(value=0.0, layout=Layout(width='auto'),  
              style=ProgressStyle(bar_color='black'))
```

--- STEP 1 METRICS ---

Original Records: 38,428,703

Records Dropped (Cancelled Trains): 595,914

Null Delays Eliminated via Drop: 595,914

Null Delays Remaining to Impute: 1,278,357

```
[4]: print("\n=== STEP 2: ORIGIN IMPUTATION (CLASS-SPECIFIC MEDIAN) ===")  
  
# 1. Calculate the median departure delay for each train  
con.execute("""  
CREATE OR REPLACE TEMP TABLE TrainOriginMedian AS  
SELECT train_no, MEDIAN(delay) AS median_dep_delay  
FROM delay_step1  
WHERE station_no = 1 AND delay IS NOT NULL  
GROUP BY train_no  
""")  
  
# 2. Print a sample of missing origin delays and their calculated median fill  
origin_sample = q("""  
    SELECT  
        a.date,  
        a.train_no,  
        a.station_name,  
        a.delay AS raw_delay,  
        COALESCE(m.median_dep_delay, 0) AS imputed_median_value  
    FROM delay_step1 a  
    LEFT JOIN TrainOriginMedian m ON a.train_no = m.train_no  
    WHERE a.station_no = 1 AND a.delay IS NULL  
    LIMIT 5  
""")  
print("\nSample of Imputed Origin Delays (station_no = 1):")  
print(origin_sample.to_string(index=False))  
  
# 3. Execute the origin imputation  
con.execute("""  
CREATE OR REPLACE TABLE delay_step2 AS
```

```

SELECT
    a.date,
    a.train_no,
    a.station_no,
    a.station_name,
    CASE
        WHEN a.station_no = 1 AND a.delay IS NULL THEN COALESCE(m.
↳median_dep_delay, 0)
        ELSE a.delay
    END AS delay
FROM delay_step1 a
LEFT JOIN TrainOriginMedian m ON a.train_no = m.train_no
""")

# 4. Extract and print Step 2 Metrics
metrics_2 = q("""
    SELECT
        (SELECT COUNT(*) FROM delay_step1 WHERE station_no = 1 AND delay IS_
↳NULL) AS missing_origins_before,
        (SELECT COUNT(*) FROM delay_step2 WHERE station_no = 1 AND delay IS_
↳NULL) AS missing_origins_after,
        (SELECT COUNT(*) FROM delay_step2 WHERE delay IS NULL) AS_
↳total_remaining_nulls
""")

missing_origins_before = int(metrics_2['missing_origins_before'][0])
missing_origins_after = int(metrics_2['missing_origins_after'][0])

print("\n--- STEP 2 METRICS ---")
print(f"Origin Values Successfully Imputed: {missing_origins_before -_
↳missing_origins_after:,}")
print(f"Remaining Mid-Route Nulls for Step 3:_
↳{int(metrics_2['total_remaining_nulls'][0]):,}")

```

=== STEP 2: ORIGIN IMPUTATION (CLASS-SPECIFIC MEDIAN) ===

Sample of Imputed Origin Delays (station_no = 1):

date	train_no	station_name	raw_delay	imputed_median_value
2025-02-08	19110	AWR	NaN	1.0
2025-02-09	19110	AWR	NaN	1.0
2025-02-10	19110	AWR	NaN	1.0
2025-02-11	19110	AWR	NaN	1.0
2025-02-12	19110	AWR	NaN	1.0

```

FloatProgress(value=0.0, layout=Layout(width='auto'),_
↳style=ProgressStyle(bar_color='black'))

```

--- STEP 2 METRICS ---

Origin Values Successfully Imputed: 44,805

Remaining Mid-Route Nulls for Step 3: 1,233,552

```
[5]: print("\n=== STEP 3: SPATIOTEMPORAL INTERPOLATION (FORWARD-FILL) ===")

# 1. Execute the spatiotemporal interpolation
con.execute("""
CREATE OR REPLACE TABLE delay_imputed AS
SELECT
    date,
    train_no,
    station_no,
    station_name,
    LAST_VALUE(delay IGNORE NULLS) OVER (
        PARTITION BY date, train_no
        ORDER BY station_no
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS delay
FROM delay_step2
""")

# 2. Print a sample showing the transition from NULL to the forward-filled value
filled_sample = q("""
SELECT
    b.date,
    b.train_no,
    b.station_name,
    b.station_no,
    b.delay AS old_delay_state,
    a.delay AS forward_filled_delay
FROM delay_step2 b
JOIN delay_imputed a
    ON b.date = a.date
    AND b.train_no = a.train_no
    AND b.station_no = a.station_no
WHERE b.delay IS NULL
LIMIT 5
""")

print("\nSample of Forward-Filled Mid-Route Delays (station_no > 1):")
print(filled_sample.to_string(index=False))

# 3. Extract and print Step 3 Metrics
metrics_3 = q("""
SELECT
```

```

        (SELECT COUNT(*) FROM delay_step2 WHERE delay IS NULL) AS_
↪nulls_before_fill,
        (SELECT COUNT(*) FROM delay_imputed WHERE delay IS NULL) AS_
↪nulls_after_fill
    """)

nulls_before = int(metrics_3['nulls_before_fill'][0])
nulls_after = int(metrics_3['nulls_after_fill'][0])

print("\n--- STEP 3 METRICS ---")
print(f"Values Successfully Interpolated: {nulls_before - nulls_after:,}")
print(f"Unresolved Nulls: {nulls_after:,}")

```

=== STEP 3: SPATIOTEMPORAL INTERPOLATION (FORWARD-FILL) ===

```

FloatProgress(value=0.0, layout=Layout(width='auto'),
↪style=ProgressStyle(bar_color='black'))

```

Sample of Forward-Filled Mid-Route Delays (station_no > 1):

date	train_no	station_name	station_no	old_delay_state
forward_filled_delay				
2025-02-08	12042	RPH	6	NaN
0.0				
2025-02-08	12467	MTDB	12	NaN
4.0				
2025-02-08	12467	BOW	17	NaN
2.0				
2025-02-08	12891	VZR	5	NaN
0.0				
2025-02-08	12891	KED	19	NaN
4.0				

--- STEP 3 METRICS ---

Values Successfully Interpolated: 1,233,552

Unresolved Nulls: 0

```

[6]: print("\n=== FINAL EXPORT & CLEANUP ===")

final_rows = int(q("SELECT COUNT(*) AS c FROM delay_imputed")['c'][0])

output_file = RESULT_DIR / "combined_delay_imputed.csv"
print(f"Exporting {final_rows:,} rows to CSV...")

# Export to CSV via DuckDB's optimized COPY command
con.execute(f"COPY delay_imputed TO '{output_file.as_posix()}' (HEADER,
↪DELIMITER ',',)")

```

```

print(f" -> Successfully saved to results/{output_file.name}")

# Drop intermediate staging tables to optimize the database size
con.execute("DROP TABLE delay_step1")
con.execute("DROP TABLE delay_step2")

```

```

=== FINAL EXPORT & CLEANUP ===
Exporting 37,832,789 rows to CSV...

FloatProgress(value=0.0, layout=Layout(width='auto'),
style=ProgressStyle(bar_color='black'))

-> Successfully saved to results/combined_delay_imputed.csv

```

[6]: <_duckdb.DuckDBPyConnection at 0x2f082fe22f0>

```

[7]: # Define the results directory
RESULT_DIR = Path.cwd().parent / "results"

# File paths (using the exact singular naming conventions from previous steps)
files = {
    "Train Dimension": RESULT_DIR / "dim_train_resolved.csv",
    "Station Dimension": RESULT_DIR / "dim_station_resolved.csv",
    "Schedule Fact": RESULT_DIR / "combined_schedule_imputed.csv"
}

print("=== SMALL DATASETS (PANDAS VERIFICATION) ===")
for name, path in files.items():
    if path.exists():
        df = pd.read_csv(path)
        print(f"\n--- {name} ({path.name}) ---")
        print(f"Total Rows: {len(df):,}")

        # Check for completeness (missing values)
        null_counts = df.isnull().sum()
        total_nulls = null_counts.sum()
        if total_nulls == 0:
            print("Missing Values: 0 (Perfect completeness)")
        else:
            print("Missing Values Detected:")
            print(null_counts[null_counts > 0].to_string())

        # Print basic uniqueness/categorical stats for string columns
        categorical_cols = df.select_dtypes(include=['object']).columns
        if len(categorical_cols) > 0:
            print(f"Unique Values in Categorical Fields:")
            for col in categorical_cols:
                print(f" - {col}: {df[col].nunique():,} unique values")

```

```

else:
    print(f"\n--- {name} ---")
    print(f"File not found: {path.name}")

print("\n=====")
print("=== MASSIVE DELAY DATASET (DUCKDB VERIFICATION) ===")

delay_csv = RESULT_DIR / "combined_delay_imputed.csv"

if delay_csv.exists():
    # Connect to an in-memory DuckDB instance to query the CSV directly
    con = duckdb.connect()

    # Query basic statistical descriptions and completeness
    stats_query = f"""
SELECT
    COUNT(*) AS total_tuples,
    COUNT(*) FILTER (WHERE delay IS NULL) AS missing_delays,
    COUNT(DISTINCT train_no) AS unique_trains,
    COUNT(DISTINCT station_no) AS unique_station_sequences,
    ROUND(AVG(delay), 2) AS mean_delay,
    ROUND(STDDEV(delay), 2) AS stddev_delay,
    MIN(delay) AS min_delay,
    MAX(delay) AS max_delay
FROM read_csv_auto('{delay_csv.as_posix()}')
"""

    delay_stats = con.execute(stats_query).df()

    print(f"\n--- Delay Fact ({delay_csv.name}) ---")
    print(f"Total Rows: {int(delay_stats['total_tuples'][0]):,}")
    print(f"Missing Delay Values: {int(delay_stats['missing_delays'][0]):,}")
    print(f"Unique Trains Tracked: {int(delay_stats['unique_trains'][0]):,}")
    print(f"Max Station Sequence:↳{int(delay_stats['unique_station_sequences'][0]):,}")
    print("\nDelay Column Statistics (in minutes):")
    print(f"  - Mean:    {delay_stats['mean_delay'][0]}")
    print(f"  - StdDev:  {delay_stats['stddev_delay'][0]}")
    print(f"  - Min:     {delay_stats['min_delay'][0]}")
    print(f"  - Max:     {delay_stats['max_delay'][0]}")

    con.close()
else:
    print(f"File not found: {delay_csv.name}")

=== SMALL DATASETS (PANDAS VERIFICATION) ===

--- Train Dimension (dim_train_resolved.csv) ---

```

```

Total Rows: 8,720
Missing Values: 0 (Perfect completeness)
Unique Values in Categorical Fields:
  - train_name: 6,455 unique values
  - primary_type_code: 8 unique values
  - raw_type_codes: 13 unique values

--- Station Dimension (dim_station_resolved.csv) ---
Total Rows: 9,027
Missing Values: 0 (Perfect completeness)
Unique Values in Categorical Fields:
  - station_name: 9,027 unique values
  - station_full_name: 8,941 unique values
  - station_zone: 20 unique values
  - station_address: 6,997 unique values

--- Schedule Fact (combined_schedule_imputed.csv) ---
Total Rows: 172,112
Missing Values: 0 (Perfect completeness)
Unique Values in Categorical Fields:
  - station_name: 8,629 unique values
  - arrival_time: 1,440 unique values
  - departure_time: 1,440 unique values

=====
=== MASSIVE DELAY DATASET (DUCKDB VERIFICATION) ===

FloatProgress(value=0.0, layout=Layout(width='auto'),
  style=ProgressStyle(bar_color='black'))

--- Delay Fact (combined_delay_imputed.csv) ---
Total Rows: 37,832,789
Missing Delay Values: 0
Unique Trains Tracked: 7,032
Max Station Sequence: 121

Delay Column Statistics (in minutes):
  - Mean: 36.86
  - StdDev: 284.15
  - Min: -120.0
  - Max: 525592.0

```

[]: